

目 录

一、引言

- 1.1 编写目的
- 1.2 软件概述
- 1.3 运行环境
- 1.4 术语与缩略语
- 1.5 游戏全流程说明
- 1.6 游戏元素与操作方法

二、软件总体设计

- 2.1 软件需求概括
- 2.2 总体架构设计
- 2.3 模块划分与关系
- 2.4 场景与界面系统设计
- 2.5 主循环与资源加载设计

三、核心模块详细设计

- 3.1 游戏入口与平台适配模块
- 3.2 村口主界面与目标导航
- 3.3 弹弓摊与资源产出模块
- 3.4 村民图鉴与喊人收集模块
- 3.5 手艺养成与三阶进化模块
- 3.6 星级与手艺上限模块
- 3.7 村庄等级与上场人数模块
- 3.8 关卡、波次与星评模块
- 3.9 布阵与分路几何模块
- 3.10 战斗引擎与自动战斗模块
- 3.11 结算、复活与广告模块
- 3.12 存档与云同步模块

四、数据结构设计

五、数据接口设计

六、出错处理设计

七、性能优化设计

八、结论

一、引言

1.1 编写目的

编写本设计说明书是深圳幸运呱科技有限公司村口大战外星人小游戏软件 V1.0.0软件著作权登记材料的一部分。本文档用于说明本软件的功能范围、总体架构、核心模块、数据结构、接口设计、异常处理和性能优化方案，证明本软件为独立开发完成的原创游戏软件。

本文档面向软件著作权审查人员及后续维护人员，重点描述软件技术实现，不包含运营数据、用户隐私数据和商业敏感策略。

1.2 软件概述

村口大战外星人是一款基于微信小游戏运行环境开发的村民收集与分路自动塔防游戏。外星人降落在县城，村口闲人翻出五金店和废品堆的破烂往身上焊；人手不够就去村委会大喇叭喊一嗓子，再叫一个来。玩家在村口攒人、喂手艺、排三路四格，开打后自动战斗；局外用弹弓摊打靶换资源。

本软件的主要功能包括：

- * 村口经营系统: 管理废铁、零件、工分、村庄经验四条资源，并给出下一步目标。
- * 弹弓摊系统: 消耗弹子打靶，按权重掉落经验、废铁、零件或工分，支持广告补弹。
- * 村民收集系统: 二十名村民按五门路四定位组成完整方阵，工分喊人入伙或加星。
- * 手艺与进化系统: 手艺档位决定数值，三档与六档切换二阶、三阶立绘和攻击方式。
- * 布阵系统: 三路四格共十二格，玩家从底部坞拎人上场，开打后锁死。
- * 自动战斗系统: 确定性 tick 引擎驱动分路阻挡、跨列支援、漏怪与超时判定。
- * 关卡与星评系统: 八十章共四百关，按漏怪、倒人和清场时间评一至三星。
- * 存档同步系统: 使用本地 Storage 与 CloudBase HTTP 后端进行存档同步，按平台隔离账号数据。

1.3 运行环境

环境项	要求说明
客户端平台	微信小游戏；后续可扩展到抖音、TapTap、H5 等平台
开发语言	TypeScript、JavaScript
渲染框架	PixiJS 7 / Canvas / WebGL 运行环境
设计分辨率	750 x 1334 竖屏，按宽度等比缩放
构建工具	npm、TypeScript 5、Vite 6，产物为单文件 IIFE bundle
后端平台	腾讯云 CloudBase 云函数 + HTTP 访问服务
数据存储	微信 Storage / 浏览器 localStorage / CloudBase 文档数据库

1.4 术语与缩略语

术语	含义说明
门路	村民与敌人的克制属性，共五条循环
定位 / 力	挨拦打修四种站位，对外收成抗打治三种力
手艺	村民的数值档位，1 到 75；视觉二三阶嵌在 3 / 6
星级	重复喊人得到的成长，管手艺上限，最多十星
弹子	弹弓摊的次数，不是货币，不进资源条
漏怪	敌人走过底线；满三只判负
CloudSync	本地存档与云端存档之间的同步机制
JWT	JSON Web Token，用于后端鉴权和平台账号识别
GAME_KEY	游戏代号 cunkou，用于集合名、token、存档 key 隔离

1.5 游戏全流程说明

玩家从启动到回村口形成一条闭环。下面按真实游玩顺序说明每一步看到什么、决定什么、系统记什么。截图按此顺序编号，补图后说明书即成为图文对照的全流程材料。

第一步 启动与健康游戏忠告

打开小游戏后先进入全屏加载页。画面展示游戏名称、加载进度条、著作权人，以及《健康游戏忠告》全文。此阶段同时预热云同步、预加载村口与战场贴图。最短展示 900 毫秒。忠告文字必须可读，属于出版审查必查项。



图1 启动加载界面 - 游戏名称、进度条与健康游戏忠告(截图占位)

第二步 进入村口

加载完成后进入村口。顶部四条资源一目了然；路中间站着已入伙的人；左侧弹弓摊、中间关卡木牌

和下一步目标、右侧图鉴墙、底部出村铁门。玩家在这里只做一件事:决定下一步去摊子、图鉴还是出村。



图2 村口主界面 - 资源条、村民、弹弓摊、关卡木牌、图鉴墙与出村铁门(截图占位)



图12 下一步目标 - 能喂 / 能喊 / 推图 / 打利索(截图占位)



图11 关卡木牌带星评 - 当前关卡与最高星(截图占位)

第三步 弹弓摊打靶换资源

点摊子进入弹弓摊。消耗一发弹子打靶，按权重掉落村庄经验、废铁、零件或工分。弹子靠离线回复

、通关补给和广告补充。工分有保底，面板写还差几发。打完回到村口，经验条和资源数字会变。



图3 弹弓摊界面 - 靶面、弹子、工分保底与广告补弹(截图占位)

第四步 图鉴、喊人与喂料

点图鉴墙进入二十人名单。已入伙的能点进去看三阶立绘并喂手艺；没见过的暗着。工分够六就点「喊一嗓子」:前四次必出新人，之后可能加星。喂料花废铁和零件，三档、六档换立绘和打法。



图4 村民图鉴 - 已入伙/没见过、定位筛选与喊人入口(截图占位)



图6 喊人入伙反馈 - 新人入伙或熟人加星(截图占位)



图5 单个村民三阶详情 - 立绘对照与喂料消耗(截图占位)

第五步 出村布阵

点铁门进入当前关。开战前先看敌方主门路，再从底部坞把人拎到三路四格上。谁站第一格谁先挨，空路会直接漏。上一关的排法会带过来，可以改。点开战后面板锁死，进入自动战斗。



图7 布阵界面 - 三路四格、底部坞与敌方门路提示(截图占位)

第六步 自动战斗、复活与结算

敌人从上往下走，村民自动出手。地面怪被本路最前排挡住，飞碟点后排。漏满三只弹出复活；看广告可以继续打。打完出结算:星评、废铁、弹子，可选广告翻倍。回村口后木牌更新关卡和星，目标木牌指向下一件该做的事。



图8 战斗进行中 - 分路进场、自动出手、漏怪与底线(截图占位)



图9 漏怪复活弹窗 - 倒下村民、漏怪数与广告复活(截图占位)



图10 通关结算 - 星评、废铁、弹子与广告翻倍(截图占位)

闭环

结算给的废铁和弹子回到村口，继续打靶、喊人、喂料、出村。推过的关可以重打补星。云同步在后台进行，换设备登录后进度仍在。

1.6 游戏元素与操作方法

资源与次数

名称	性质	来源	去处
废铁	货币	通关结算为主，摊子为零头	喂手艺
零件	货币	弹弓摊破电视、蓝筐	喂手艺，卡先喂谁
工分	货币	摊子喇叭与保底	喊人，每次 6
村庄经验	公共等级	摊子铁皮罐、铁盆	升村庄等级，涨全员面板和上场人数
弹子	次数，不是货币	离线回复、通关、失败、广告	只打靶，不能买不能换

二十名村民

门路	抗	拦	打	治
站远点打	锅盖二哥	渔网婶	弹弓叔	喇叭爷
挨得住	铁柱	石磨姨	棉裤张	二舅
下手重	秤砣老赵	王大锤	电锯哥	杀猪匠
越挨越猛	高压锅婶	钢板哥	屠户老李	鞭炮婆
带一帮人	牵狗大爷	养鸡婶	三婶	保温壶爷

操作方法

操作	手势	结果
出村	点底部铁门	进入当前关布阵
打靶	点摊子后点发射	耗一发弹子，掉资源
喊人	图鉴页点喊一嗓子	耗 6 工分，新人入伙或熟人加星
喂料	详情页点喂	耗废铁和零件，手艺 +1
上阵	坞里向上拖到格子	该格站上这个人
换位 / 下场	场上拖到另一格或拖回坞	改排法
开战	点开战	锁阵，自动战斗
复活	漏满 3 只后点广告复活	漏怪清零，继续打

回村	结算或布阵点回村口	回到 home
----	-----------	---------

本软件不提供键盘操作。所有交互均为触屏点按与拖拽。真机安全区避开微信 / 抖音右上角胶囊按钮；顶部资源条与标题对齐胶囊垂直中心。

二、软件总体设计

2.1 软件需求概括

本软件采用模块化、数据驱动的设计方式。客户端负责游戏循环、界面渲染、输入交互、本地存档和资源加载；后端负责平台登录、JWT 签发、云端存档读写、并发版本校验和跨平台账号隔离。战斗规则集中在 BattleEngine，渲染层和数值模拟器共用同一套 tick，避免两套算法。

软件核心需求包括以下几个方面：

- * 提供稳定的分路自动塔防，包括布阵、阻挡、跨列支援、漏怪、超时和星评。
- * 提供村民收集与养成进度，包括喊人入伙、手艺喂料、三阶进化、星级上限和村庄等级。
- * 提供局外资源循环，包括弹弓摊打靶、弹子回复、通关结算和广告补弹。
- * 提供可靠的本地存档与云存档，保证退出、断网、重进及不同设备之间的存档一致性。
- * 提供平台适配，使微信、抖音和 H5 复用同一套游戏规则。

2.2 总体架构设计

软件整体架构可划分为入口层、平台适配层、核心服务层、规则层、场景表现层、资源层和后端服务层。各层之间通过明确的接口协作，降低具体平台 API、渲染对象和业务逻辑之间的耦合度。

```
src/main.ts
+-- core/Game.ts           Pixi 初始化、设计分辨率、安全区
+-- core/PlatformService.ts 微信/抖音/H5 平台能力适配
+-- core/PersistService.ts 本地持久化与云同步白名单管理
+-- core/BackendService.ts  HTTP 登录、pull、push 请求封装
+-- core/CloudSyncManager.ts 启动拉取、防抖上传、冲突覆盖
+-- core/RunMemory.ts      养成存档读写与业务动作
+-- game/BattleEngine.ts   战斗规则唯一真源
+-- balance/*              村民、关卡、摊子、村庄纯数据
+-- scenes/VillageScene.ts 村口四页: home / stall / folks / one
+-- scenes/BattleScene.ts  布阵与战斗渲染
+-- cloudfunctions/cunkou-api CloudBase HTTP 后端服务
```

2.3 模块划分与关系

模块层	代表文件/对象	功能简述
入口层	main.ts / Game.ts	完成游戏初始化、主循环、生命周期和启动同步
平台层	PlatformService	封装 request、storage、login、广告、生命周期

服务层	PersistService / BackendService	封装本地存储、云同步快照、后端 HTTP
规则层	BattleEngine / RunMemory / balance	战斗、养成、关卡、摊子的唯一规则
场景层	VillageScene / BattleScene	绘制界面、处理点击和反馈
浮层	SettleOverlay / ReviveOverlay / Loading	结算、复活、加载与健康忠告
资源层	TextureLoader / SfxPlayer / BgmPlayer	贴图、音效、背景音乐加载
后端层	cunkou-api	平台登录、JWT 鉴权、存档拉取与上传

模块之间采用"入口调度、服务抽象、规则层持有状态、场景层订阅并触发业务"的协作方式。场景层不直接操作平台 API 和后端接口，而是通过 RunMemory 或核心服务完成，保证各平台复用同一套游戏规则。战斗渲染层只读 BattleEngine 的状态和事件，不自己算伤害。

```
main.ts -> 初始化 Pixi、TextureLoader、SceneManager、云同步预热
VillageScene -> 调用 RunMemory 方法 -> 修改养成存档
BattleScene -> BattleEngine.tick -> 消费 BattleEvent 放特效
PersistService.subscribe(changedKeys) -> CloudSyncManager.scheduleSync(reason)
BackendService.request(path, body) -> PlatformService.request -> CloudBase HTTP
```

2.4 场景与界面系统设计

软件采用两个主场景加若干浮层的结构。VillageScene 用四个页面覆盖局外全部功能，刻意不做编队、废品站、门路研发等多入口:home 出村，stall 打靶，folks 图鉴与喊人，one 看三阶再喂料。BattleScene 覆盖布阵与自动战斗。结算、复活、加载由独立 Overlay 管理。

界面	入口	玩家在这里决定什么
加载	启动	无决策，展示健康游戏忠告
村口 home	加载完成 / 回村	下一步去摊子、图鉴还是出村
弹弓摊 stall	村口摊子	打哪一发、要不要看广告补弹
图鉴 folks	村口图鉴墙	喊谁、筛哪一种力
详情 one	图鉴点人	喂不喂、升完长什么样
布阵	出村铁门	谁上场、站哪一路哪一格
战斗	开战按钮	观战；漏怪时可选择复活
结算	胜负判定	看星评、拿废铁和弹子、是否翻倍

2.5 主循环与资源加载设计

游戏启动时先挂 LoadingScreenOverlay，展示插画、进度条和健康游戏忠告，同时预热 CloudSyncManager，并按清单预加载村口与战场贴图。最短展示 900 毫秒，避免一闪而过。资源就绪后再切换到 VillageScene。

主循环由 Pixi Ticker 驱动。村子页按需重绘；战场按 100 毫秒逻辑步长调用 BattleEngine.tick，渲染层插值位置并消费事件数组播放特效与音效。切后台时触发立即存档与云端 flush。

三、核心模块详细设计

3.1 游戏入口与平台适配模块

游戏入口模块完成 Pixi 应用初始化、舞台创建、安全区计算、加载页挂载、启动期云同步等待、生命周期事件绑定等工作。平台适配模块通过 PlatformService 统一封装微信、抖音和 H5 的差异，包括 storage、request、login、onHide、onShow、激励视频等能力。业务代码禁止直接写 typeof wx / typeof tt。

```
PlatformService.request(options) -> wx.request / tt.request / fetch
PlatformService.getStorageSync(key) -> 平台同步存储
PlatformService.login() -> wx.login / tt.login
PlatformService.onHide(callback) -> 切后台保存与云同步
PlatformService.showRewardedVideo(adUnitId) -> 激励视频 Promise
```

平台适配模块采用能力检测方式选择具体实现:抖音环境优先绑定 tt 对象，微信环境绑定 wx 对象，浏览器环境使用 fetch 和 localStorage。GameKey 按平台加后缀:微信 cunkou，抖音 cunkou_tt，避免两个宿主串档。

3.2 村口主界面与目标导航

村口是全部功能的中枢。顶部资源条常驻废铁、零件、工分、村庄经验，大数用万/亿格式化。路中间站已入伙村民；左侧弹弓摊、中间关卡木牌与下一步目标、右侧图鉴墙、底部出村铁门。nextGoal 按"能喂 > 能喊 > 推图 > 打利索"排序，任何时候点开村子都有下一步。

```
nextGoal(mem):
  if someone can afford nextFeed: return { kind: craft, short: "能喂" }
  if mem.credits >= CALL_COST: return { kind: call, short: "能喊" }
  if stageTop not yet cleared: return { kind: stage, short: "推图" }
  if any cleared stage has stars < 3: return { kind: stars, short: "打利索" }
  return { kind: done, short: "齐了" }
```



图12 下一步目标 - 能喂 / 能喊 / 推图 / 打利索(截图占位)



图11 关卡木牌带星评 - 当前关卡与最高星(截图占位)

3.3 弹弓摊与资源产出模块

弹弓摊是局外唯一的资源出口。玩家消耗一发弹子打靶，系统按千分权重抽取靶位，掉落村庄经验、废铁、零件或工分。弹子是次数不是货币:不进资源条、不能买卖、没有第二个去处。

靶位	解锁村庄等级	主产出	特殊规则
铁皮罐 x3	1	村庄经验	权重最高，摊子主产出
绿酒瓶 x2	1	废铁	废铁只是零头，大头在通关
破电视	1	零件	零件的唯一摊出来源
蓝塑料筐	2	经验+零件	两头都给的中靶
吊着的铁盆	6	经验	打中免费再来一发
挂着的旧喇叭	8	工分	最小最偏，另有保底

弹子来源包括:离线按间隔回复(满仓封顶)、通关补给、失败也给一发、广告每日三次各五发。工分另做保底:若干发必出一次，面板显示还差几发，避免玩家以为喇叭永远打不中。打中一个靶会按规则

碰倒相邻靶，相邻按五成结算经验和废铁。

```
shootStall(nowMs):
  mem = settlePellets(nowMs)
  if mem.pellets <= 0: return undefined
  result = shoot(rng, mem.villageLv, mem.stallPity)
  add villageExp / scrap / parts / credits from result.gain
  mem.pellets -= 1
  persist(mem)
```



图3 弹弓摊界面 - 靶面、弹子、工分保底与广告补弹(截图占位)

3.4 村民图鉴与喊人收集模块

村民池固定二十人，按五门路乘四定位组成完整方阵。启动时 assertRosterComplete 校验每格恰好一人，否则"这关克你的坦克、换另一条门路的坦克上来"会变成无解。图鉴页已入伙的能点进去喂，没见过的暗着；抗、打、治三种力可筛选。

门路	克谁	代表村民
站远点打	带一帮人	锅盖二哥、渔网婶、弹弓叔、喇叭爷
挨得住	下手重	铁柱、石磨姨、棉裤张、二舅
下手重	越挨越猛	秤砣老赵、王大锤、电锯哥、杀猪匠
越挨越猛	站远点打	高压锅婶、钢板哥、屠户老李、鞭炮婆
带一帮人	挨得住	牵狗大爷、养鸡婶、三婶、保温壶爷

喊人消耗六工分。前四次必出尚未入伙的新人，让新玩家几天内就有一队能排；之后进入重复池。喊到熟人折六十废铁并给全队星最低的人加一星。星满后再喊只折废铁。

```
callVillager(nowMs):
  if mem.credits < CALL_COST: return undefined
  res = rollCall(progress, callCount+1, rng, allIds, STAR_MAX)
  if res.isNew: roster.push(res.id); seenIds.add(res.id)
  else if res.starTo: stars[res.starTo] += 1
  credits -= 6; scrap += res.scrap
```



图4 村民图鉴 - 已入伙/未见过、定位筛选与喊人入口(截图占位)



图6 喊人入伙反馈 - 新人入伙或熟人加星(截图占位)

3.5 手艺养成与三阶进化模块

手艺是村民的数值档位。面板 hp / atk 乘 craftMul:前十档用手校表，十档之后每档加百分之六复利，上限七十五档。视觉阶由手艺推导:不到三档为一阶，三到五档为二阶，六档起为三阶。每一阶必须有独立立绘，并且改攻击方式或射程，不许只调数字。

打法 kind	出现阶	效果
plain	一阶默认	定位默认:奶就奶、拦就减速
regen / standUp	铁柱等	周期性回血；倒下一次能原地起来
reflect / cleave	锅盖、电锯等	反伤；横扫最多三人
slowHard / pierce	渔网、弹弓等	强减速；穿透后续目标
lifesteal / burst	屠户、高压锅等	吸血；血过线炸一圈
laneHeal / allHeal / hasteAura	喇叭、保温壶	整路奶、全场奶、加速光环

喂料消耗废铁和零件。前五档主要吃废铁，后段吃零件，因为推图期废铁来自首通结算，通关后零件从摊子持续堆积。十档之后的价格按百分之八复利续。料不够、星卡住或已经焊满时按钮不可用。

```
buyEvo(id):
  cost = nextFeed(progress, id)
  if !cost or scrap < cost.scrap or parts < cost.parts: return
  craft[id] += 1
  evo[id] = evoFromCraft(craft[id]) // 1 / 3 / 6 -> 视觉 1 / 2 / 3
  persist
```



图5 单个村民三阶详情 - 立绘对照与喂料消耗(截图占位)

3.6 星级与手艺上限模块

星级不是村庄公共等级，而是指定村民的成长。重复喊人时加到当前星最低的人，避免把所有星堆在同一个主力上把节奏冲垮。星管手艺上限:零星只能喂到三档(二阶)，两星解开三阶，五星才能焊满前十档，十星才能走到七十五档。每星给面板百分之八的乘数。

星级	手艺上限	设计目的
0-1	3	新人先看到二阶立绘，三阶要再喊到他
2-4	6 / 8	解开三阶，中段还能再喂两档
5	10	前四十关手校段的天花板
6-10	20 / 32 / 45 / 60 / 75	四百关主线新接的跑道

3.7 村庄等级与上场人数模块

村庄经验只有一条公共条，不逐个村民喂经验。等级只放大全员 hp / atk，每级百分之三，不碰射程和出手间隔。前二十级经验表手校，二十级之后按百分之六复利。二十级之后产出乘 yieldMul，每级百分之五，避免后期喂料跑几千天。

村庄等级	上场人数	说明
1	3	开局挨、拦、打各一人

2 / 4 / 7	4 / 5 / 6	与前四十关曲线对齐
11 / 15 / 20	8 / 10 / 12	到二十级站满十二格
21-120	12	人数封顶，后面只长数值和产出

3.8 关卡、波次与星评模块

主线八十章共四百关。前八章的地名、敌人组合和台词全部手写；第九章起用地名池和六种敌人循环生成，难度分两段:前八段用手校倍率，后面用更平坦的尾段倍率，避免把前四十关的节奏冲掉。开战前必须把敌方主门路显示给玩家，玩家据此换人。

外星人	门路	特点
方块	挨得住	慢、肉，前期顶路
小灰	带一帮人	快、成群
铁罐	挨得住	高防，要砸壳
突击	越挨越猛	速度快、下手重
飞碟	站远点打	飞行，不被阻挡，点后排
装甲	下手重	最肉，后期墙

星评看三件事:漏了几个、倒了几个人、清得利索不利索。没赢是零星；漏过是一星；一个没漏但有人倒下或超时是二星；一个没漏、一个没倒、并且在 par 时间内清完才是三星。通关后仍可重打补星，木牌显示最高星。

```
rateStars(leaked, fallen, won, elapsedMs, parMs):
  if !won: return 0
  if leaked > 0: return 1
  if fallen > 0: return 2
  return elapsedMs <= parMs ? 3 : 2
```

3.9 布阵与分路几何模块

战场逻辑是三路四格。四格落在轴坐标 2 / 3 / 4 / 5，敌人挡点在 1.5，底线在最后一格再往下一格。地面怪被本路最靠前的活人挡住；飞碟不被阻挡，专打本路最后排。空路直接漏。打和修能支援左右邻列，邻列距离加一格代价，本列仍然优先；挨和拦只守本路。

布阵阶段玩家从底部坞把人拎到格子上。开打后位置锁死。下一关默认铺上一次的排法，只有全新存档才用 autoPlace 兜底，避免第一眼看到空棋盘。不做推荐阵容一键上。

```
laneReachOf(role):
  return role in {tank, block} ? 0 : 1

pickFoe(fighter):
  candidates = foes in range, |laneGap| <= reach
  sort by same-lane first, then closer pos
```

```
return first
```

3.10 战斗引擎与自动战斗模块

BattleEngine 是战斗规则的唯一真源。渲染层和 formulas/simulate 都驱动同一个 tick。引擎是确定性的:同样的布阵和关卡，结果永远一样，变化量交给布阵和养成，不交给骰子。一步顺序是:出怪、敌人动、村民动、收尾判定。刚走进射程的敌人当帧就能被打到。

判定	条件	结果
漏怪	敌人 pos 超过底线	leaked += 1，满 3 只可触发复活或失败
超时	超过 timeLimitMs	判负
清场	波次放完且场上无活着的敌人	按 rateStars 给星
阻挡	地面怪遇到本路最前排活人	停在 BLOCK_POS 开打
飞碟	flying = true	不停步，点本路最后排

```
tick(state, dtMs = TICK_MS):
    spawnDueEnemies(state)
    moveFoes(state)
    villagersAct(state) // 选目标、出手、奶、特效 kind
    applyLeaksAndTimeout(state)
    return events[] // 渲染层只消费事件，不算伤害
```



图8 战斗进行中 - 分路进场、自动出手、漏怪与底线(截图占位)

3.11 结算、复活与广告模块

漏满三只时弹出 ReviveOverlay。玩家可看激励视频复活一次(日限两次)，复活后漏怪计数清零并继续打。不复活或次数用尽则进入失败结算。失败也给一发弹子，避免空手回村。

胜利结算展示星评、本关废铁和弹子。首次通关废铁五十(乘产出倍率)，重打十五；首次通关额外多给弹子。广告可将本关废铁翻倍，日限五次。解锁下一关，并把最高星写入 stageStars。

```
settleStage(stageId, won, stars):
    pellets = won ? PELLET_CLEAR + (first ? PELLET_FIRST : 0) : PELLET_LOSE
```

```
scrap = won ? round((first ? 50 : 15) * yieldMul(villageLv)) : 0
if won: stageTop = max(stageTop, stageId+1)
stageStars[stageId] = max(old, stars)
```



图9 漏怪复活弹窗 - 倒下村民、漏怪数与广告复活(截图占位)



图10 通关结算 - 星评、废铁、弹子与广告翻倍(截图占位)

3.12 存档与云同步模块

养成进度集中在 RunMemory，字段包括村庄等级、名单、手艺、星级、四条资源、弹子、关卡进度、星评和上次布阵。存档格式版本 REV 不匹配直接重置，不硬折上一版字段。写盘走 PersistService，命中白名单则标记 dirty。

CloudSyncManager 负责启动期拉取、数据合并、1.5 秒防抖上传、失败指数退避和切后台强制上传。后端 cunkou-api 提供 /login、/save/pull、/save/push、/health。登录按平台 code 换 openid，签发含 userId、platform、gameKey 的 JWT。若本地 updatedAt 落后，后端返回 STALE_UPDATE，客户端下行覆盖，防止旧设备回写。

```
scheduleSync(reason):
  if !cloudReady: markPending
  clearPreviousTimer()
  setTimeout(() => pushSave(exportCloudSnapshot()), 1500ms)
```

```
pushSave(snapshot):
POST /cunkou-api/save/push Authorization: Bearer token
body = { schemaVersion, updatedAt, payload, clientFingerprint }
```

四、数据结构设计

4.1 本地养成存档

```
RunMemory = {
  rev: 3,
  villageLv: number, villageExp: number,
  roster: string[],
  evo: Record<id, 1|2|3>,
  craft: Record<id, 1..75>,
  stars: Record<id, 0..10>,
  scrap, parts, credits: number,
  pellets, pelletAtMs, stallPity, adCount, adDay,
  stageId, stageTop, stageStars: Record<stageId, 0..3>,
  layout: { id, lane, cell }[],
  callCount, seenIds: string[]
}
```

4.2 云同步数据结构

```
cunkou_playerData = {
  _id: string,
  userId: 'wx:openid' | 'dy:openid' | 'tap:id' | 'anon:uuid',
  platform: 'wx' | 'dy' | 'tap' | 'anon',
  schemaVersion: number,
  updatedAt: number,
  clientFingerprint: string,
  payload: Record<string, string>,
  payloadKeys: string[],
  lastWriteAt: number
}
```

4.3 云同步白名单

只有白名单内的数据会打包上云。token、匿名设备 id、广告日限、调试开关仅保留本地。当前白名单只有一条养成主存档，按平台加前缀:cunkou_run_memory / cunkou_tt_run_memory。

存储 key	说明
cunkou_run_memory	微信养成主存档
cunkou_tt_run_memory	抖音养成主存档
cunkou_token / cunkou_anon_id	仅本地，不上云

cunkou_ad_day	广告日限，仅本地
---------------	----------

4.4 战斗与关卡结构

```
Placement = { villager, lane, cell, evoStage, stars, craft? }
Fighter = { uid, def, lane, cell, pos, hp, atk, range, interval, alive, ... }
Foe = { id, def, lane, pos, hp, atk, slowMs, alive }

StageDef = {
  id, chapter, index, label, name, pitch,
  hpMul, atkMul, waves, timeLimitMs, parMs, mainLane, suggestLv
}
SpawnGroup = { lane, enemy, count, gapMs, atMs }
```

4.5 村民与摊子配置结构

```
VillagerDef = {
  id, name, lane, role, job, flavor,
  evo: [{ name, pitch }, { name, pitch }, { name, pitch }]
}
TargetDef = {
  id, name, weight, exp, scrap, parts, credits, rebound?, pitch
}
```

五、数据接口设计

5.1 本地存储接口

本地存储通过 PlatformService 封装 wx.setStorageSync、wx.getStorageSync、wx.removeStorageSync 等接口。H5 环境则使用 localStorage 作为兼容实现。

```
PersistService.readRaw(key): string | null
PersistService.writeRaw(key, value, options)
PersistService.remove(key, options)
PersistService.exportCloudSnapshot(): PersistSnapshot
PersistService.importCloudSnapshot(snapshot)
```

5.2 后端 HTTP 接口

接口	方法	功能
/cunkou-api/login	POST	平台登录、生成 userId、签发 JWT
/cunkou-api/save/pull	POST	按当前 userId 拉取云端存档
/cunkou-api/save/push	POST	上传云端存档，按 updatedAt 处理版本冲突
/cunkou-api/health	GET/POST	健康检查

5.3 账号隔离接口

后端将不同平台账号映射为带前缀的 `userId`，例如 `wx:openid`、`dy:openid`、`tap:userId`、`anon:uuid`。JWT payload 中包含 `sub`、`plt`、`gk` 字段，用于校验用户、平台和游戏代号，防止跨游戏或跨平台串档。经分上报永远用 `BASE_GAME_KEY=cunkou`，不把 `cunkou_tt` 传给分析 SDK。

5.4 规则层对外接口

```
loadMemory() / persist(next)
shootStall() / claimAdPellets() / callVillager() / buyEvo(id)
setStageId(id) / saveLayout(slots) / settleStage(id, won, stars)
createBattle(stage, placements) / startFight() / tick() / placeAt()
statsOf(def, evoStage, stars, villageLv, craft?)
```

六、出错处理设计

6.1 网络异常处理

客户端请求后端时设置十秒超时。登录、拉取、上传失败不会阻塞本地游玩，而是记录日志并继续使用本地存档。`CloudSyncManager` 对上传失败进行计数，并按指数退避延迟下一次同步；连续失败后进入低频重试模式。启动同步最多等待 2.5 秒，超时按 `cache-only` 继续进村。

6.2 数据冲突处理

云端保存时检查 `updatedAt`。若服务端已有更新版本，则返回 `STALE_UPDATE` 和远端数据；客户端收到后下行覆盖本地，避免旧设备把新设备进度回写。`payload` 大小默认限制 256KB。

6.3 资源加载异常处理

`TextureLoader` 在贴图未就绪时返回空纹理，场景层退回色块绘制，不挡玩。战场和村子都按"贴图缺失仍可操作"设计，避免单张资源失败导致白屏。

6.4 存档异常处理

`RunMemory` 读取时校验 `REV`。版本不匹配直接走空档，不迁移上一版字段。名单、布阵、星评会过滤未知村民 `id` 和越界坐标；数值字段做上下限夹取。JSON 解析失败视为无档。

6.5 战斗与输入异常处理

布阵只接受名单内、格子未占用的人；重复拖同一人会先从旧格拿起。引擎对死亡单位、空目标、越界 `lane` 做跳过，不抛异常到渲染层。全局 `onError` / `onUnhandledRejection` 会打日志并上报经分，不中断主循环。

七、性能优化设计

7.1 渲染与对象复用

设计分辨率 750x1334，舞台按宽度等比缩放，长屏用 `logicHeight` 把底栏落到安全区，避免悬空。战场单位用 `UnitActor` 复用精灵，特效由 `CombatFx` 统一调度，避免每帧 `new` 对象。村子四页按打开页重绘，不在后台页保留完整显示列表。

7.2 战斗步进与模拟

逻辑步长 100 毫秒，足够表达攻速差异，又不会让千局回归变慢。战斗事件用数组而不是回调，模拟器一秒可跑数千 tick，且同样输入同样输出。护栏测试只扫描玩家真正打到过的关，避免对未到达的后期关做无效断言。

7.3 存档防抖与写入优化

本地存档使用同步写入，避免异步 `setStorage` 顺序不确定造成旧数据覆盖新数据。云同步采用 1.5 秒防抖，短时间内多次喂料、打靶合并为一次上传。切后台时使用 `flushNow` 立即上传。连续失败按 1.5 秒起跳、上限 30 秒退避。

7.4 后端负载优化

后端接口仅保存白名单存档 payload，不上传 token 和广告日限；同时限制存档大小 256KB。通过 `userId` 唯一索引定位用户存档，pull/push 均为单文档读写，适合小游戏高频轻量同步场景。

八、结论

深圳幸运呱科技有限公司村口大战外星人小游戏软件 V1.0.0 围绕村民收集、手艺进化、分路布阵和自动战斗建立了完整的软件架构。软件客户端、规则层、资源、存档和后端服务均按模块化方式设计，具备清晰的数据结构、接口边界和异常处理机制，满足软件著作权登记文档鉴别材料对技术说明的要求。